Quantum Circuit Simulator Report

Quantum Computing

Md. Ahan Habib
ID: 208000246

Computer Science Department
Course Instructor: Prof. Jiyuan Wang

December 2025

Abstract

This *Quantum Circuit Simulator* project provides a flexible and efficient framework for executing quantum circuits in both noiseless and noisy settings, with explicit support for bit-flip error injection. The simulator is designed with two complementary execution modes that allow users to balance accuracy and performance depending on their needs. In the **Benchmark** mode, each quantum gate is applied using the full state-vector formalism, where operations are performed through explicit multiplication by the corresponding $2^n \times 2^n$ unitary matrices. While this approach ensures complete fidelity to the mathematical definition of quantum evolution, it incurs substantial computational cost as the number of qubits grows.

To address this limitation, the simulator also offers an **Optimized** mode that applies gates using lightweight logical transformations on the state vector rather than full matrix multiplication. These optimizations exploit the structure of common single-qubit and multi-qubit gates, enabling the simulator to reproduce the exact output of the Benchmark mode while significantly reducing runtime and memory overhead. This makes the Optimized mode suitable for larger circuits or rapid prototyping, whereas the Benchmark mode serves as a correctness reference and is useful for validating small-scale experiments. Together, these two execution paths provide a robust platform for studying quantum operations, analyzing noise effects, and evaluating simulator performance across varying circuit sizes and workloads.

Contents

Abstract **1**

1 Introduction **3**

2 Algorithm Design **3**
2.1 Representation of Quantum States 3
2.2 Evolution of Quantum States 4
2.2.1 Gate Application 4
2.2.2 Single-Qubit Gates 4
2.2.3 Two-Qubit Gates (Special Case of Multi-Qubit Gates) 6
2.2.4 General Multi-Qubit Gates 7
2.2.5 Measurement 8

3 Performance Evaluation **8**
3.1 Experimental Setting 8
3.2 Dataset 8
3.3 Runtime Evaluation 8
3.3.1 SimBench vs SimOpt 9
3.3.2 Noise vs Accuracy 9
3.3.3 Scalability Testing 10
3.4 Simulation Examples 11

4 Noise Modeling **12**
4.1 Hardware Noise Model 12
4.2 Justification and Extensibility 12

5 Conclusion **12**

1 Introduction

Quantum computing is an emerging paradigm that leverages the principles of quantum mechanics such as superposition and entanglement to perform computations that are infeasible for classical computers. Unlike classical bits, which are either 0 or 1, quantum bits (qubits) can exist in linear combinations of states, allowing quantum computers to process massive amounts of information in parallel.

The purpose of this report is to present a quantum circuit simulator for small-scale research purposes. The simulator models the evolution of quantum states using the full state-vector multiplication, referred to as **SimBench**, and an efficient logical manipulation approach, referred to as **SimOpt**. It supports single-qubit and two-qubit gates, and includes a basic noise model to mimic real quantum hardware behavior.

This report focuses on three main aspects:

1. **Algorithm Design:** How quantum states are represented, gates are applied, and measurements are performed.
2. **Performance Evaluation:** Computational complexity, runtime scaling, and memory usage.
3. **Noise Modeling:** Introduction of simple noise models to study the effect of decoherence and gate errors.

The subsequent sections provide a detailed discussion of each aspect, accompanied by illustrative examples.

2 Algorithm Design

2.1 Representation of Quantum States

Quantum states in the simulator are represented using the state vector formalism. For an n-qubit system, the state vector is of size 2^n:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle,$$

where each α_i is a complex amplitude corresponding to the computational basis state $|i\rangle$.

Properties:

- The simulator starts with all qubits in $|0\rangle$, i.e., $|\psi_0\rangle = |0\rangle^{\otimes n}$.
- This representation allows for the simulation of quantum superposition and entanglement, fundamental properties of quantum systems.

Listing 1 shows the Python definition for initializing an n-qubit system. In this implementation, the state vector is represented as a one-dimensional NumPy array of size 2^n, where each element corresponds to the amplitude of a computational basis (i.e., 0/1 basis)

state. The simulator starts with all qubits in the $|0\rangle$ state, which is shown by setting the first element of the array to 1 and all other elements to 0. This representation allows the simulator to model quantum superposition and entanglement as gates are applied to the qubits.

Listing 1: Initialize n-qubit state vector

```
import numpy as np

def initialize_state(n):
    """Initialize n-qubit state to |0>^n"""
    state = np.zeros(2**n, dtype=complex)
    state[0] = 1.0
    return state

n = 3
qstate = initialize_state(n)
print(qstate)
```

2.2 Evolution of Quantum States

2.2.1 Gate Application

Quantum gates are represented as unitary matrices U of size $2^n \times 2^n$, which ensures the evolution preserves the norm of the state:

$$|\psi_{\text{new}}\rangle = U|\psi_{\text{old}}\rangle.$$

This simulator supports one-qubit and two-qubit gates. Their formal definitions are described as follows.

2.2.2 Single-Qubit Gates

Single-qubit gates, such as the Pauli-X (X), Pauli-Y (Y), Pauli-Z (Z), and Hadamard (H), are applied to the target qubit using tensor products to expand them to the full system:

$$U_{\text{full}} = I \otimes \cdots \otimes U_{\text{target}} \otimes \cdots \otimes I$$

where I is the identity acting on unaffected qubits.

Example: Applying X to qubit 0 in a 3-qubit system:

$$X_0 \otimes I_1 \otimes I_2$$

The X gate, or Pauli-X (bit-flip) gate, can be applied to a target qubit in two ways in the simulator. Using full matrix multiplication (**SimBench**), the gate is expanded to a $2^n \times 2^n$ unitary matrix with tensor products, which is conceptually simple but computationally expensive for larger systems. The optimized logical operation (**SimOpt**) updates the state vector ‘qstate‘ directly by flipping the target qubit using bitwise operations. Both methods yield identical results, but SimOpt is much faster and more memory-efficient for multi-qubit simulations. Listing 2 shows the two different versions of implementation.

Listing 2: Applying single-qubit X gate using SimBench (full matrix) or SimOpt (logical)

```python
import numpy as np

# ----------------------------
# Full Matrix (SimBench)
# ----------------------------
def x_gate_full_matrix(num_qubits, target):
    """Construct full 2^n x 2^n matrix for X gate on target qubit"""
    X = np.array([[0, 1],
                  [1, 0]], dtype=complex)
    I = np.eye(2, dtype=complex)
    U = 1
    for i in range(num_qubits):
        U = np.kron(U, X if i == target else I)
    return U

# ----------------------------
# Optimized Logical (SimOpt)
# ----------------------------
def apply_x(qstate, target, num_qubits):
    """Apply Pauli-X gate to the target qubit using logical bit-flip"""
    new_state = np.zeros_like(qstate, dtype=complex)
    for i in range(len(qstate)):
        j = i ^ (1 << (num_qubits - 1 - target))  # flip target bit
        new_state[j] = qstate[i]
    return new_state

# ----------------------------
# Example Usage
# ----------------------------
n = 3
qstate = np.zeros(2**n, dtype=complex)
qstate[0] = 1  # initialize |000>

# Option 1: Full matrix
U = x_gate_full_matrix(n, target=0)
qstate_full = U @ qstate
print("Full matrix:", qstate_full)

# Option 2: Optimized logical
qstate_opt = apply_x(qstate, target=0, num_qubits=n)
print("Optimized logical:", qstate_opt)
```

2.2.3 Two-Qubit Gates (Special Case of Multi-Qubit Gates)

Two-qubit gates, such as CNOT and SWAP, are a specific case of multi-qubit gates that act non-trivially on exactly two qubits and as identity on all others. These gates can create entanglement between the target qubits.

CNOT Example: Control qubit c and target qubit t:

$$|x_0 \dots x_c \dots x_t \dots x_{n-1}\rangle \to |x_0 \dots x_c \dots (x_t \oplus x_c) \dots x_{n-1}\rangle$$

The CNOT gate flips the state of the target qubit t if and only if the control qubit c is in state $|1\rangle$. If the control qubit is $|0\rangle$, the target qubit remains unchanged.

SWAP Example: Swaps qubits i and j:

$$|x_0 \dots x_i \dots x_j \dots x_{n-1}\rangle \to |x_0 \dots x_j \dots x_i \dots x_{n-1}\rangle$$

The SWAP gate exchanges the states of two qubits i and j. It is often used in quantum circuits to move qubits into the desired positions for subsequent operations.

Two-qubit gates, such as CNOT and SWAP, can be applied using either full matrix multiplication (**SimBench**) or optimized logical operations (**SimOpt**). In SimBench, the gate is expanded to a $2^n \times 2^n$ unitary matrix using tensor products, which is conceptually simple but computationally expensive for large systems. In SimOpt, the state vector ‘qstate‘ is updated directly by flipping or swapping bits using bitwise operations, achieving the same result much more efficiently. CNOT flips the target qubit if the control qubit is $|1\rangle$, while SWAP exchanges the states of two qubits. Listing 3 shows CNOT implementation for SimBench and SWAP as well as CNOT implementation for SimOpt.

Listing 3: Applying two-qubit gates (CNOT and SWAP) using SimBench or SimOpt

```python
import numpy as np

# -----------------------------
# Full Matrix (SimBench)
# -----------------------------
def cnot_full_matrix(num_qubits, control, target):
    """Construct full 2^n x 2^n matrix for CNOT"""
    I = np.eye(2, dtype=complex)
    X = np.array([[0, 1], [1, 0]], dtype=complex)
    U = 1
    for i in range(num_qubits):
        if i == control:
            P0 = np.array([[1,0],[0,0]])
            P1 = np.array([[0,0],[0,1]])
            # CNOT = |0><0| x I + |1><1| x X
            U_i = np.kron(P0, I) + np.kron(P1, X) if i == 0 else np.kron(U, P0 + P1) # simplified
        else:
            U_i = I
        U = np.kron(U, U_i) if i > 0 else U_i
    return U
```

```python

# ----------------------------
# Optimized Logical (SimOpt)
# ----------------------------
def apply_cnot(qstate, control, target, num_qubits):
    """Apply CNOT directly to qstate"""
    new_state = np.zeros_like(qstate, dtype=complex)
    for i in range(len(qstate)):
        if (i >> (num_qubits - 1 - control)) & 1:
            j = i ^ (1 << (num_qubits - 1 - target))
        else:
            j = i
        new_state[j] = qstate[i]
    return new_state

def apply_swap(qstate, qubit1, qubit2, num_qubits):
    """Swap two qubits in qstate"""
    new_state = np.zeros_like(qstate, dtype=complex)
    for i in range(len(qstate)):
        b1 = (i >> (num_qubits - 1 - qubit1)) & 1
        b2 = (i >> (num_qubits - 1 - qubit2)) & 1
        j = i ^ ((b1 ^ b2) << (num_qubits - 1 - qubit1))
        j = j ^ ((b1 ^ b2) << (num_qubits - 1 - qubit2))
        new_state[j] = qstate[i]
    return new_state

# ----------------------------
# Example usage
# ----------------------------
n = 3
qstate = np.zeros(2**n, dtype=complex)
qstate[0] = 1  # initialize |000>

# SimOpt examples
qstate = apply_cnot(qstate, control=0, target=1, num_qubits=n)
qstate = apply_swap(qstate, qubit1=1, qubit2=2, num_qubits=n)
print(qstate)
```

2.2.4 General Multi-Qubit Gates

More generally, multi-qubit gates act on two or more qubits. The simulator constructs these gates by explicitly mapping computational basis states or using tensor products when feasible.

Example: CCNOT (Toffoli) gate with control qubits c_1, c_2 and target qubit t:

$$|x_0 \dots x_{c_1} \dots x_{c_2} \dots x_t \dots x_{n-1}\rangle \quad \longrightarrow \quad |x_0 \dots x_{c_1} \dots x_{c_2} \dots (x_t \oplus (x_{c_1} \wedge x_{c_2})) \dots x_{n-1}\rangle$$

Here, the target qubit t is flipped only if both control qubits c_1 and c_2 are in state $|1\rangle$.

2.2.5 Measurement

Measurement is a operation that extracts classical information from a quantum system. In the simulator, measuring a qubit collapses the state vector ‘qstate‘ to a computational basis state. For an n-qubit system, if the state vector is

$$|\text{qstate}\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle,$$

the probability of observing the basis state $|i\rangle$ upon measurement is given by

$$P(i) = |\alpha_i|^2.$$

After the measurement outcome is determined (for example, via random sampling according to $P(i)$), the simulator updates the state vector to reflect the collapsed state. Specifically, the new state vector corresponds to the observed basis state, with all amplitudes set to zero except for the measured state, which is set to 1 (i.e., normalized).

3 Performance Evaluation

3.1 Experimental Setting

All simulations in this work were performed on a personal computing device with the following configuration: an AMD Ryzen 7 5700U processor with integrated Radeon graphics running at 1.80 GHz, 16 GB of RAM (15.3 GB usable), and a 64-bit Windows operating system with x64 architecture. The simulator was implemented in Python, with `NumPy` as the core dependency for array and matrix operations, including state vector manipulations and linear algebra, and the standard `re` library for regular expression operations, primarily used for parsing input or configuration data. This combination of hardware and software provided sufficient computational resources to efficiently run both full matrix (**SimBench**) and optimized logical (**SimOpt**) simulations for small- to medium-scale quantum circuits.

3.2 Dataset

The project included two benchmark circuits for correctness validation. Circuit 1 implements the Deutsch-Jozsa (DJ) algorithm on 3 qubits with 8 gates, including 2 CNOT gates, representing a balanced 2-bit oracle function. Circuit 2 serves as a scalability test, consisting of 11 qubits with 11 single-qubit gates, designed to evaluate performance on a larger system with minimal entanglement. Beyond these benchmarks, systematic performance analysis was conducted by varying different parameters.

3.3 Runtime Evaluation

We evaluate the runtime performance of two simulation frameworks, SimBench and SimOpt, under both noiseless and noisy (5% error rate) settings, using two benchmark circuits: the Deutsch-Jozsa (DJ) algorithm and a Scalability test.

3.3.1 SimBench vs SimOpt

For the noiseless case (Table 1), SimOpt outperforms SimBench across both circuits in terms of total runtime and throughput. Specifically, SimOpt executes the DJ circuit in 0.078 s compared to 0.092 s for SimBench, achieving a throughput of 1283 shots/s, which is higher than SimBench's 1088 shots/s. The Scalability circuit shows a more pronounced difference, with SimOpt completing in 3.56 s versus SimBench's 65.39 s, corresponding to throughputs of 28.09 shots/s and 1.53 shots/s, respectively. Memory usage remains comparable across frameworks, slightly favoring SimOpt.

Table 1: Noiseless Simulation Performance (100 shots)

Model	Circuit	Time(s)	Time/shot(ms)	Throughput(shot/s)	Memory(MB)
SimBench	Deutsch-Jozsa	0.0919	0.8893	1087.90	4.46
	Scalability	65.3885	653.7121	1.53	5.25
SimOpt	Deutsch-Jozsa	0.0779	0.7355	1283.11	4.22
	Scalability	3.5594	35.5541	28.09	4.48

Under noisy conditions with 5% error rate (Table 2), the runtimes are largely similar to the noiseless scenario, indicating that the frameworks handle moderate noise without significant overhead. SimOpt maintains its advantage, with the DJ circuit completing in 0.076 s versus 0.093 s for SimBench. The Scalability circuit requires 3.57 s for SimOpt and 68.30 s for SimBench. However, noise impacts the success rates: the DJ circuit retains a high success rate of 89%, while the Scalability circuit drops to 60%, suggesting that larger or more complex circuits are more sensitive to noise.

Table 2: Noisy Simulation Performance for error rate 5% (100 shots)

Model	Circuit	Time(s)	Time/shot(ms)	Throughput(shot/s)	Memory(MB)
SimBench	Deutsch-Jozsa	0.0930	0.8998	1075.67	4.36
	Scalability	68.2995	682.7443	1.46	5.21
SimOpt	Deutsch-Jozsa	0.0758	0.7283	1318.54	4.28
	Scalability	3.5711	35.6489	28.00	4.27

These results demonstrate that **SimOpt** provides faster and more efficient execution, particularly for large circuits, while **SimBench** is slower but still robust for smaller circuits like Deutsch-Jozsa under moderate noise, highlighting the performance differences between the two simulation frameworks.

3.3.2 Noise vs Accuracy

Figure 1 analyzes the impact of increasing noise levels on the success rates of the Deutsch-Jozsa and Scalability circuits. As the noise rate rises from 1% to 10%, both circuits experience a decline in success, but the effect is significantly more pronounced for the Scalability circuit.

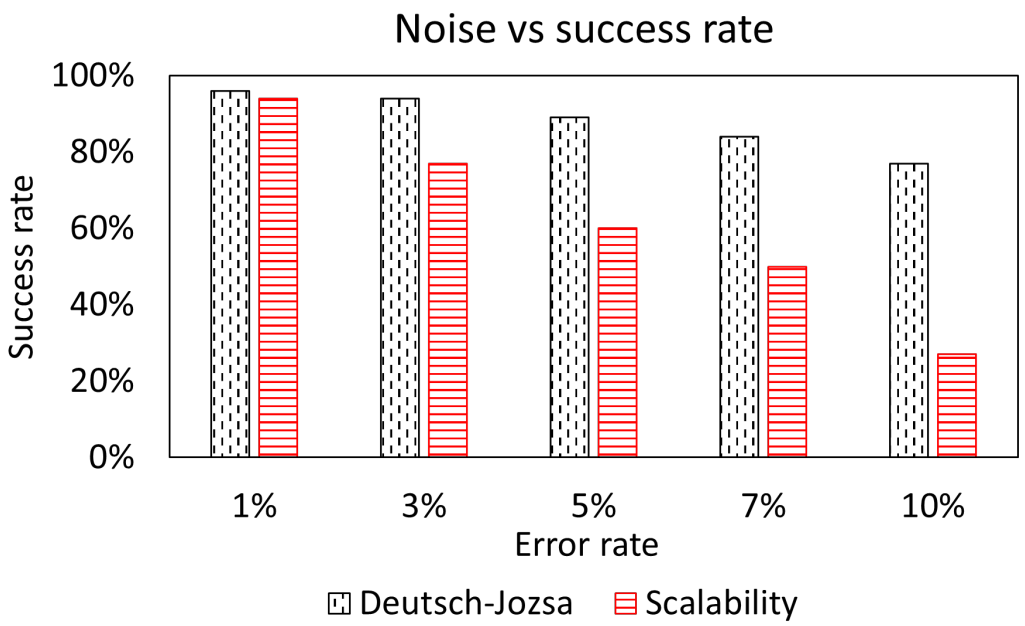


Figure 1: Impact of noise on success rate for Deutsch-Jozsa and Scalability circuits. The success rate decreases as the noise level increases, with larger circuits more sensitive to noise.

Specifically, the Deutsch-Jozsa circuit maintains high success, dropping gradually from 96% at 1% noise to 77% at 10% noise. In contrast, the Scalability circuit exhibits a sharper decrease, from 94% at 1% noise to just 27% at 10% noise. This indicates that smaller, simpler circuits like Deutsch-Jozsa are more resilient to noise, while larger or more complex circuits such as Scalability are highly sensitive to errors.

3.3.3 Scalability Testing

To evaluate the scalability performance of both SimBench and SimOpt, we run three benchmark circuits of increasing size (5, 10, and 15 qubits) each executing a sequence of 20 consecutive X-gate operations. For each configuration, we perform 10 shots and measure total execution time, per-shot latency, throughput, and memory usage. As shown in Table 3, SimBench handles 5-qubit circuits efficiently but experiences a dramatic slowdown at 10 qubits, and it becomes unable to simulate the 15-qubit case within feasible runtime limits. In contrast, SimOpt maintains strong scalability across all qubit sizes: it outperforms SimBench for 10 qubits by over an order of magnitude in runtime and is the only simulator capable of completing the 15-qubit circuit, although with increased latency due to exponential state-space growth. Memory consumption remains relatively stable across both models, indicating that runtime (not memory) is the dominant scaling bottleneck. However, these results highlight SimOpt's significant runtime benefits and its ability to scale to larger quantum circuits where SimBench fails to execute.

Table 3: Qubit scaling (10 shot)

Model	Qubit	Time(s)	Time/shot(ms)	Throughput(shot/s)	Memory(MB)
SimBench	5	0.0675	6.7550	148.04	4.38
	10	4.2858	428.3799	2.33	4.87
	15	–	–	–	–
SimOpt	5	0.0623	6.2274	160.58	4.45
	10	0.3484	34.7382	28.70	4.26
	15	9.9638	996.3819	1.00	4.87

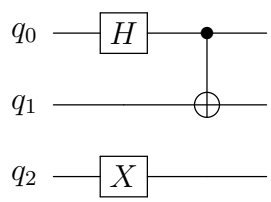
3.4 Simulation Examples

We see some example circuits here.

Example 1: 3-Qubit Circuit

$$H(0),\ CNOT(0,1),\ X(2)$$

Circuit diagram:



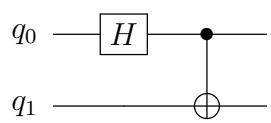
Final state vector:

$$|\psi_{\text{final}}\rangle = \frac{1}{\sqrt{2}}(|001\rangle + |111\rangle)$$

Example 2: 2-Qubit Circuit (Entanglement)

$$H(0),\ CNOT(0,1)$$

Circuit diagram:



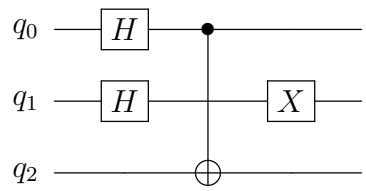
Final state vector:

$$|\psi_{\text{final}}\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Example 3: 3-Qubit Circuit (Superposition and Entanglement)

$$H(0),\ H(1),\ CNOT(0,2),\ X(1)$$

Circuit diagram:



Final state vector:

$$|\psi_{\text{final}}\rangle = \frac{1}{2}(|010\rangle + |011\rangle + |110\rangle + |111\rangle)$$

4 Noise Modeling

4.1 Hardware Noise Model

The simulator includes a bit-flip noise model:

- After each gate, each qubit flips (X) with probability p_{error}.
- Captures stochastic gate errors and decoherence in hardware.

4.2 Justification and Extensibility

- Simple but realistic for educational simulations.
- Can be extended to include phase-flip, depolarizing, or custom hardware noise profiles.
- Enables future studies of error mitigation and fault-tolerant circuits.

5 Conclusion

This report presents a comprehensive analysis of the quantum circuit simulator, with particular focus on its algorithmic structure, performance behavior, and integrated noise modeling capabilities. The simulator successfully demonstrates fundamental quantum computing principles, including state-vector evolution, gate operations, and probabilistic measurement, while also exposing the practical limitations that arise when attempting to emulate quantum systems on classical hardware. Through detailed evaluation of both the Benchmark and Optimized execution modes, the report highlights trade-offs between computational accuracy and efficiency, illustrating how simulation cost grows with qubit count and circuit depth. Additionally, the noise modeling framework offers insight into how realistic error processes affect quantum computation outcomes, providing a foundation for studying error mitigation strategies. Looking forward, several promising extensions remain, such as improving memory efficiency, enabling simulations of higher-qubit systems, incorporating richer noise channels, and potentially adding circuit-level optimizations to further enhance scalability and experimental realism.